

Python 代码助手

面向 Python 学习者的本地优先 AI 编程助手

产品介绍 · 对外版

版本：1.1.0

日期：2026-08-31

密级：公开

一、产品概述

「Python 代码助手」是一款面向 Python 初学者与自学者的智能编程辅助插件，运行于 PyCharm 与 IntelliJ 平台。它把编程学习中最高频、最耗时的几类任务——读懂代码、写出规范代码、排查运行报错、沉淀学习笔记——统一交给大语言模型在本地完成，让使用者在编辑器内就能获得即时、准确、可回看的帮助。

与常见云端编程助手不同，本产品坚持「本地优先」：所有个人数据只保存在使用者自己的电脑上，AI 能力通过使用者自己的密钥调用，不上传、不收集、不依赖任何远程服务器即可使用全部核心功能。

二、目标用户与核心价值

2.1 目标用户

- Python 初学者 / 自学者：需要即时解释代码、纠正不规范写法。
- 在校学生：需要把零散的报错与心得整理成可复习的学习笔记。
- 需要快速排查报错的开发者：希望一键看懂报错、一键得到修复建议。

2.2 核心价值

价值	说明
价值	说明
即时反馈	输入即检测，错误早发现，边写边改
即时反馈	输入即检测，错误早发现，边写边改
一句话求助	解释、注释、修复、问答，选中代码即可发起
一句话求助	解释、注释、修复、问答，选中代码即可发起
知识沉淀	把每次报错与心得变成可导出、可复习的笔记
知识沉淀	把每次报错与心得变成可导出、可复习的笔记
隐私优先	数据本地化、密钥自理，敏感信息不出本机
隐私优先	数据本地化、密钥自理，敏感信息不出本机

三、产品架构

产品采用清晰的「三层」分层结构，职责分明、易于理解：

- 交互层：IDE 插件与网页工作台，负责「你点哪里、看什么结果」。

- 服务层：本地 AI 服务，仅在本机运行，负责接收请求、调度模型、读写本地数据。
- 智能层：大语言模型 API，负责真正的理解与生成。

数据流动遵循一条原则：插件与网页工作台只与本机服务通信，对模型的调用与结果整理全部收敛在本机服务内完成，个人数据仅在本机读写。这样的设计让敏感信息始终停留在使用者自己的设备上，也为离线基础能力留出了空间——即便暂时无法联网调用模型，内置的规则引擎仍能提供基础纠错提示。

四、核心功能详解

4.1 输入时智能检测

使用者在编辑器中输入代码时，插件会在短暂停顿后自动检查常见错误与不规范写法，并在代码旁以内联提示的方式给出「问题 → 建议」。提示由内置规则与模型分析共同提供，即使未配置密钥也能使用基础的规则检查。

4.2 代码理解与生成

- 解释代码：用中文把选中代码「在做什么」讲清楚，帮助初学者读懂逻辑。
- 生成注释：为选中代码自动补充逐行中文注释，规范书写习惯。

4.3 报错处理

- 报错解释：粘贴运行时报错，AI 解释错误原因与修改方法。
- 自动修复：找出选中代码中的错误并给出修复后的代码。

4.4 AI 问答与搜索

- AI 问答：直接向模型提问 Python 相关问题。
- 中文搜函数：用中文关键词查找 Python 函数、库与用法。
- 函数对比：对比两个函数或指令的差异与适用场景。

4.5 本地文件库与学习笔记

「记录错误代码」「保存代码片段」等功能把内容沉淀到本机文件库。使用者可多选、整理，一键让 AI 把错误记录生成结构化学习笔记（错误代码、正确写法、原因分析、举一反三），并导出为 PDF 或 Markdown 供复习与分享。

4.6 网页端工作台

除了编辑器内的插件交互，产品还提供一个网页工作台，集中呈现 AI 问答、中文搜索、学习笔记与函数对比，与插件共享同一份本地数据，适合在更大画布上整理与回顾。

五、本地报错知识库（亮点）

5.1 设计理念：让 AI 回答「有据可依」

通用大模型在回答报错问题时，容易给出看似合理实则空泛的建议。本产品为此内置了一套本地报错知识库，让 AI 在回答前先检索真实、可查的历史报错案例，使回答更贴近实际工程经验、减少凭空猜测。

5.2 双库结构

- 公共库（只读）：由公开社区沉淀的高质量报错案例与对应解决方案，作为共享知识底座。
- 个人库（可增删改查）：使用者自己的报错记录，随时维护、只属于本人。

两个库共用同一套检索能力，公共库保证「有料可查」，个人库保证「越用越懂你」。

5.3 检索增强问答（RAG）

当使用者发起报错解释、自动修复或相关问答时，系统会先在本地知识库中检索最相关的历史案例，把其中的报错类型、错误信息与真实解决方案作为参考材料一并交给模型。模型据此生成回答，既保留通用能力，又获得真实案例的支撑。整个过程在本地完成，检索结果不离开使用者的设备。

5.4 一键扩充

产品支持从公开数据源批量导入高质量报错解决方案（数据来自社区公开内容、遵循其开源许可）。一条命令即可完成下载、清洗、入库、索引构建与自检，让知识库从零快速积累到可用的规模，且随时可重复、可增量。

5.5 价值小结

维度	价值
维度	价值
准确性	用真实案例约束模型输出，减少幻觉与空话
准确性	用真实案例约束模型输出，减少幻觉与空话
个性化	个人库持续沉淀，越用越贴合自己的场景
个性化	个人库持续沉淀，越用越贴合自己的场景
可扩展	公共库可按需扩充，规模与覆盖自由调节
可扩展	公共库可按需扩充，规模与覆盖自由调节
隐私性	检索与问答全程本地，数据不出设备
隐私性	检索与问答全程本地，数据不出设备

六、安全与隐私

6.1 数据本地化

学习笔记、错误记录、代码片段与知识库数据全部保存在使用者本机，系统默认不向任何云端同步。个人数据的主权完全归使用者本人。

6.2 密钥自理

AI能力通过使用者自己的密钥调用。密钥由使用者本人填写并保存在本机，系统不代管、不收集、不写入任何可被分发或公开的内容中。每一位使用者都使用自己的密钥、为自己的用量负责。

6.3 最小暴露

本机服务仅面向使用者本人使用，不对外提供访问入口；网页工作台也必须经由插件发起才能进入。从设计上，可被复制、分发、暴露的内容中都不存放任何密钥或敏感信息。

七、快速上手

三步即可开始使用：

- 第一步：安装插件。在 PyCharm / IntelliJ 的插件设置中，从本地文件安装产品插件包，重启 IDE。
- 第二步：填入密钥（一次即可）。在插件的设置入口填入自己申请的 AI 服务密钥，保存在本机。
- 第三步：开始使用。选中代码后通过右键菜单或快捷键发起「解释 / 注释 / 修复 / 报错解释」，或打开网页工作台进行问答与笔记整理。

若希望启用本地报错知识库的检索增强能力，可再执行一次数据扩充命令，之后 AI 回答报错问题时会自动带上知识库参考。

八、系统要求

项目	要求
项目	要求
运行环境	PyCharm / IntelliJ IDEA（较新版本）
运行环境	PyCharm / IntelliJ IDEA（较新版本）
本地服务	Python 3.10 及以上
本地服务	Python 3.10 及以上
AI 能力	使用者自备大模型服务密钥（按需）
AI 能力	使用者自备大模型服务密钥（按需）
联网	核心 AI 功能需联网调用模型；基础规则检测可离线

九、总结

「Python 代码助手」把「看懂、写对、修好、记住」这一整条学习闭环，装进了一个本地优先的编辑器插件里。它以即时检测与一键求助降低入门门槛，以本地文件库与学习笔记承载沉淀，以本地报错知识库与检索增强提升回答质量，并在全程坚持数据本地化与密钥自理，让使用者学得放心、用得安心。

面向初学者，它是一位随叫随到的
AI 助教；面向开发者，它是一个本地、可扩展、越用越懂你的报错知识系统。